

Ostbayerische Technische Hochschule Amberg-Weiden
Department of Electrical Engineering, Media and Computer Science

Autonomous Project in Webots Simulation

Autonomous Robotics

Team members

Poovallal Jayepalan
Ebong Njume
Somantha Manuranga
Adarsh Sarda

Supervised by

Prof. Dr. Thomas Nierhoff

July 10, 2026

1 Approach

1.1 Overview

The objective of this project is to control a four-wheel Rosbot autonomously in previously unknown Webots mazes. Each maze contains a blue pillar and a yellow pillar. The mission order implemented by the controller is to locate and reach the blue pillar first and then to locate and reach the yellow pillar. Green floor regions are treated as hazards and must not be crossed. The environments also contain ordinary floor-standing walls and, in some mazes, elevated wooden planks that may intersect the robot body while remaining outside the LiDAR scan plane.

The controller does not receive a pre-built map. It estimates the robot pose from wheel encoders and a compass, constructs a two-dimensional occupancy grid from sensor measurements, selects exploration targets, plans routes, and follows the planned routes while continuing to update perception. Exploration and target execution therefore overlap: a pillar can be localized while the robot is still extending the map.

A central implementation challenge is that the 2D LiDAR may pass underneath an elevated plank. The code supplements LiDAR mapping with depth-camera measurements and wooden-colour segmentation. Depth points are retained only within a configured vertical interval that overlaps the robot body, and an optional LiDAR-comparison filter suppresses depth points that are also observed by the LiDAR. These retained points are treated as likely elevated obstacles and inserted into the same occupancy grid used by the planner.

1.2 Strategy overview

1. **Perception and localization:** update odometry every simulation step and periodically process RGB and depth data for pillars, green regions, and wooden obstacles.
2. **Exploration:** select a frontier cluster at the boundary between currently known free cells and unknown cells. If no sufficiently large frontier is detected, select a random candidate waypoint and attempt to plan toward it.
3. **Global planning and local control:** use A* on an inflated occupancy grid, then follow the resulting path with a look-ahead steering controller whose heading correction is computed by a PID controller.
4. **Recovery:** react to mapped or live-sensor hazards, replan after collisions, detect prolonged low progress, and temporarily relocate when a pillar route cannot be found.

1.3 Software architecture and control loop

The implementation is separated into the following modules: `robot_controller.py` for Webots devices, odometry, and obstacle measurements; `vision.py` for HSV-based image processing; `map_manager.py` for the occupancy grid, frontier extraction, and visualization; `planner.py` for A* and B-spline path smoothing; `pid.py` for steering control; `constants.py` for configuration; and `rosbot_python_controller.py` for the mission state machine and main loop.

The Webots time step is 32 ms. During each main-loop iteration the controller performs the following operations:

1. Update position and heading using the front wheel encoders and the compass.

2. Update the map on configured update steps, subject to separate turn-rate gates for LiDAR and depth-camera data.
3. Every third step, process the heavier visual operations and cache their results for the intervening control steps.
4. Update the mission state and target waypoint.
5. Normally re-run A* every 15 main-loop steps, or immediately when recovery sets the force-replan flag.
6. Evaluate green, depth, body-sensor, and LiDAR obstacle conditions and run recovery when required.
7. Otherwise calculate and apply the left and right wheel velocities for path following.

Thus, odometry, hazard reactions, and wheel control run at the simulation-step rate, whereas vision, map updates, display updates, and A* planning are intentionally throttled by their own intervals.

1.4 Technical implementation

1.4.1 Robot, sensors, and localization

The platform is a four-wheel skid-steer Rosbot. Turning is produced by commanding different velocities to the left and right wheel pairs. The submitted controller initializes the following Webots devices:

- A 360-degree 2D LiDAR for floor-level obstacle mapping and directional collision checks.
- An RGB camera for HSV colour segmentation of blue, yellow, green, and wooden regions.
- A depth camera for distance estimation, height filtering, pillar range estimation, and projection of coloured regions into the map.
- Four short-range distance sensors. The two front sensors contribute to collision detection, while the two rear sensors are checked before and during backing recovery.
- Front-left and front-right wheel encoders and an IMU compass for pose estimation.

At start-up, encoder references are stored and the initial map position is set to $(0, 0)$. Translational displacement is estimated from the average wheel travel, while the compass directly supplies the heading. The position update is

$$x_k = x_{k-1} + \Delta s \cos(\theta_k), \qquad y_k = y_{k-1} + \Delta s \sin(\theta_k) \tag{1}$$

where Δs is the average of the left and right wheel displacement. This is odometry, not a full probabilistic localization system; position error can still accumulate because of wheel slip and modelling error.

Table 1: Sensor use in the submitted implementation.

Feature	LiDAR	RGB	Depth	Short-range
Floor-standing walls	main	–	filtered support	–
Likely elevated walls	comparison	wood mask	main	–
Green hazards	–	main	range/projection	–
Blue/yellow pillars	–	main	range	–
Collision response	main	–	main	front/rear

1.4.2 Coordinate transformations

Measurements are first expressed relative to the robot and then transformed into world coordinates using the estimated robot pose:

$$p_{\text{world}} = R(\theta)\, p_{\text{local}} + t, \qquad R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \tag{2}$$

For an RGB/depth pixel with horizontal image coordinate u and measured depth d, the implementation uses

$$x_{\text{local}} = d, \qquad y_{\text{local}} = -\frac{(u - c_x)\, d}{f_x} \tag{3}$$

with $f_x = 558.89$ pixels and $c_x = 320$ pixels. The same rotation-and-translation convention is used for pillar positions, green samples, and wooden-obstacle samples.

1.4.3 Occupancy-grid mapping

The map contains 300×300 cells and represents a $10\,\text{m} \times 10\,\text{m}$ area, giving a resolution of approximately $0.033\,\text{m}$ per cell. Each cell stores a log-odds score. Bresenham lines are traced from the robot cell to each sensor endpoint. Traversed cells receive free-space evidence and the endpoint receives occupied-space evidence:

$$L(x,y) \leftarrow L(x,y) - 0.3 \text{ for traversed cells;} \qquad L(x,y) \leftarrow L(x,y) + 0.8 \text{ for an endpoint} \tag{4}$$

The values are clipped to $[-5, 5]$. In the submitted constants, a cell is classified as occupied when $L > 1.0$ and free when $L < -0.5$; intermediate values remain unknown. A cell can therefore become provisionally occupied after repeated evidence. However, only cells at or above the free-update cap of 2.0 are protected from subsequent free-ray erosion. Permanently stamped green or wooden samples use a log-odds value of 5.0.

- **Turn gating:** LiDAR and depth-camera map writes use separate wheel-speed-difference thresholds. This reduces endpoint smearing during rotation.
- **Visited cells:** cells occupied by the robot footprint are forced to free and are protected from later green/wood stamping. This preserves evidence of the route already travelled, but it does not guarantee that A* will always recover a return path after inflation and localization error.

- **Planning cleanup:** small occupied blobs are removed, narrow gaps are closed with a morphological operation, and obstacles are inflated before A* is executed.
- **Optional map freeze:** the implementation can block all map writes after blue is reached. In the submitted constants this feature is disabled (`FREEZE_MAP_AFTER_BLUE = False`).

1.4.4 Elevated wooden-obstacle mapping

The code uses two related depth-based mechanisms. First, sampled depth-camera points are converted to physical height above the floor. Points are retained only when their estimated height is between 0.07 m and 0.20 m and their range is below 2.0 m. This rejects most floor points and obstacles that are high enough for the robot to pass underneath.

Second, the RGB image is segmented with a fixed HSV range for wooden colours. Multiple pixels are sampled from each connected wood-colour blob, associated with depth values, height-filtered, transformed into world coordinates, and stamped into the occupancy grid. Because fixed HSV thresholds are heuristic, the report treats these as likely wooden obstacles rather than guaranteeing perfect material recognition.

When `DEPTH_MAP_ONLY_LIDAR_MISSES` is enabled, a depth sample is discarded if a LiDAR point falls within the configured local-grid matching tolerance. This reduces duplicate mapping of ordinary walls; a retained sample is interpreted as a likely obstacle missed by the LiDAR, not as mathematically proven evidence of a floating wall.

The robot performs an initial approximate 360-degree rotation to localize visible pillars from different headings. In the submitted controller, map updates during this rotation are commented out to avoid rotational smearing. Therefore, the start scan must not be described as a floating-wall mapping sweep.

1.4.5 Green-hazard detection and mapping

Green regions are segmented in HSV space and grouped with connected-component analysis. Multiple pixels from each sufficiently large blob are sampled and projected through the depth image into world coordinates. Samples are accepted only within 1.5 m and stamped as occupied cells, except where the visited-cell mask indicates that the robot has already physically driven.

The closest accepted green sample near the image centre is cached. When this distance is below the configured escape threshold, `green_escape()` commands an in-place timed rotation for 35 simulation steps. The code does not measure the completed heading change and does not guarantee an exact 180-degree turn or a separate forward drive-away phase. After the rotation, the normal state machine and replanning continue.

1.4.6 Pillar detection and localization

Blue and yellow masks are produced from fixed HSV thresholds. A valid colour blob must contain at least the configured minimum number of pixels. Pillar depth is obtained from depth values under the colour mask, using a percentile to reduce contamination by farther background pixels. Each observation is transformed into a world position and blended with the stored estimate using an exponential moving average with $\alpha = 0.15$.

- During a searching state, a visible mission pillar can be centred by an in-place proportional camera-offset controller.

- When the robot is within the 2.0 m lock radius of the active pillar estimate, further position updates are suppressed to reduce short-range target jitter.
- Within the 0.7 m direct-approach radius, the controller can bypass A* and drive toward the visible mission pillar because the pillar itself may block the A* goal cell.
- Approach-via offsets are implemented but are $(0.0, 0.0)$ in the submitted constants, so they are disabled in this configuration.

1.4.7 Frontier exploration

Frontier candidates are free cells adjacent to unknown cells. The map manager groups connected candidate pixels and ignores clusters whose area is not greater than the configured minimum of 15 pixels. A valid cluster is scored using its area and distance from the robot, and the centre of the highest-scoring cluster becomes the exploration waypoint.

If no qualifying frontier is returned, the controller samples a random world-coordinate candidate in the configured range. The point is not guaranteed to be free or reachable; the later A* attempt may fail, in which case another target is selected. Likewise, the absence of a qualifying frontier does not prove that every map cell has been explored.

1.4.8 A* planning and path processing

The planner performs an eight-connected A* search. Its cost is not purely geometric distance: diagonal motion costs 1.414, orthogonal motion costs 1.0, unknown cells receive an additional penalty, and cells close to obstacles receive a distance-transform-based clearance penalty. The method therefore searches for a low-cost route according to the configured cost map rather than guaranteeing the geometrically shortest route.

- Obstacle inflation is first attempted with a radius of 4 grid cells. The fallback list may try a smaller radius when no path fits at the nominal margin. These are configurable grid margins, not an exact calculation of the robot half-width.
- If the estimated start cell is occupied after inflation, `snap_to_free_cell()` searches outward for a nearby non-zero start cell.
- The search may terminate when it reaches the exact goal or comes within the configured goal radius.
- The returned grid sequence is smoothed with a cubic B-spline when enough points are available. The spline reduces grid-like direction changes, but the smoothed points are not revalidated by a second collision test; live obstacle checks and replanning remain important.

1.4.9 Look-ahead PID path following

For a valid path, the controller selects a point several path samples ahead and computes the wrapped heading error between the robot orientation and that point. The steering command is

$$\omega = K_p e + K_i \int e\,dt + K_d \frac{de}{dt}, \qquad (K_p, K_i, K_d) = (6.0,\ 0.1,\ 1.5) \tag{5}$$

The integral term is clamped to reduce wind-up. Base speed is reduced as the absolute heading error increases, reduced again within 1.0 m of the target, and further reduced when A* required a smaller-than-nominal inflation radius. Wheel commands are $v_L = v - \omega$ and $v_R = v + \omega$. The uploaded `follow_path_steering()` function does not contain an explicit `PIVOT_IN_PLACE_ANGLE` branch; therefore, the report does not claim a separate threshold-triggered stop-and-pivot behaviour.

If the path is empty or shorter than three points, the function returns equal negative search-turn commands rather than a zero-velocity hold. At the system level, A* failure normally changes the state or target and sets immediate replanning.

1.4.10 Mission state machine and recovery

The main mission states are `SEARCHING_BLUE`, `GOING_TO_BLUE`, `SEARCHING_YELLOW`, and `GOING_TO_YELLOW`, with `DETOUR_BLUE` and `DETOUR_YELLOW` used after route failures. When A* cannot produce a route to the active pillar and the robot is not already in the final direct-approach range, the known pillar position is retained, the robot is assigned an exploration/relocation waypoint, and the pillar route is retried after the detour timer or waypoint arrival.

- **Collision escape:** the robot backs up while checking the two rear short-range sensors, then performs a timed random-direction in-place turn. LiDAR contributes to the original collision trigger but is not the rear check used inside the backup loop.
- **Low-progress detection:** every 50 steps, displacement is compared with 0.08 m. Three consecutive low-progress intervals trigger a deeper escape and immediate replanning.
- Fast wedge logic is present but effectively disabled in the submitted constants because `WEDGE_MIN_EXPECTED` is set to 9999.0 and `WEDGE_PROGRESS_FRAC` to 0.0.
- Breadcrumb recording and reverse retracing are implemented but disabled because `ENABLE_BREADCRUMB_RETR = False`.
- Map freezing after blue is implemented but disabled because `FREEZE_MAP_AFTER_BLUE = False`.

1.5 Configuration and maze-specific tuning

The uploaded submission contains one controller implementation and one uploaded constants file. The algorithms are parameterized through constants controlling mapping thresholds, inflation, look-ahead distance, speed, sensor ranges, and recovery timing. During development, these parameters were tested and adjusted for maze geometry. The report does not claim that all mazes used identical runtime values unless the same constants file was in fact used for the recorded run.

For the uploaded configuration, examples include `BASE_NAV_SPEED = 6.0`, `PURE_PURSUIT_LOOKAHEAD = 15`, `TIGHT_PATH_LOOKAHEAD = 7`, `INFLATION_RADIUS = 4`, `ASTAR_REPLAN_INTERVAL = 15`, and `PERCEPTION_INTERVAL = 3`. Disabled optional features are stated explicitly rather than presented as active behaviour.

Table 2 lists every constant that was changed from its default value for a specific maze; any constant not listed used the shared default.

Each override was chosen for a specific reason. In Maze 2, a looser turn gate keeps the map updating through the long corridors' gentle turns, an extra inflation-fallback radius eases the

Maze	Parameter	Default	Used
1	(all defaults)		
2	`MAP_SKIP_TURN_SPEED_DIFF`	4.0	5.0
	`ASTAR_INFLATION_FALLBACK`	(4,2)	(4,3,2)
	`FLOATING_WALL_MAX_RANGE`	2.0	2.5
3	`BASE_NAV_SPEED`	6.0	10.0
	`BLUE_APPROACH_VIA`	(0,0)	(0, 0.9)
4	`PURE_PURSUIT_LOOKAHEAD`	15	11
5	`BASE_NAV_SPEED`	6.0	12.0

Table 2: Per-maze parameter overrides; every other value is shared.

planner into narrow passages, and a longer floating-wall range catches planks along the long sightlines. In Maze 3, the layout is open enough to drive faster, and the approach-via point routes the robot to blue from the safe side of a plank. In Maze 4, a shorter look-ahead keeps the path-following controller closer to the centreline through the tight zig-zag corridors. In Maze 5, the layout is open, so the highest speed setting was used.

The Maze 5 value of `BASE_NAV_SPEED = 12.0` is worth flagging: `MAX_WHEEL_SPEED` clamps each wheel command at 10, and the commanded wheel speeds are $12 \pm \omega$, so any steering term smaller than 2 leaves both wheels saturated at 10 and yields no differential turning. In effect, this setting introduces a small steering deadband rather than simply increasing speed. It functioned correctly on Maze 5's open layout, but a setting of `BASE_NAV_SPEED = 10.0` would give the same straight-line speed while keeping steering authority intact, and should be verified against Maze 5 if this deadband behaviour is not desired.

2 Results

Table 3 reproduces the timing information available in the original report. Maze 2 contains only a recorded Start-to-Blue time; its second leg and total completion time were not provided. Consequently, the results support full recorded completion for Mazes 1, 3, 4, and 5, but not a claim that all five runs were fully recorded.

Table 3: Recorded navigation times (mm:ss).

Maze	Start to Blue	Blue to Yellow	Total
1	00:00–01:39	01:39–02:39	02:39
2	00:00–01:48	not recorded	not recorded
3	00:00–00:40	00:40–01:06	01:06
4	00:00–00:57	00:57–01:55	01:55
5	00:00–00:34	00:34–00:54	00:54

Among the four fully recorded runs, the total time ranged from 00:54 in Maze 5 to 02:39 in Maze 1. In those four runs, the Start-to-Blue leg was longer than the Blue-to-Yellow leg. This is consistent with, but does not by itself prove, the interpretation that more exploration and map construction occurred before reaching blue. Maze 2 cannot be included in the complete-leg comparison because its Blue-to-Yellow time is missing.

The implementation was designed specifically to reduce failures caused by elevated planks that are absent from the LiDAR scan. The report can state that depth/colour mapping was used

in the successful system, but the submitted timing table alone is not a controlled LiDAR-only ablation. Therefore, it does not quantify the improvement over a LiDAR-only baseline.

2.1 Limitations of the evaluation

- Only one timing result per listed run is reported; repeated-trial means, variances, and success rates are not available.
- Maze 2 is incomplete in the recorded table.
- The report does not contain a controlled comparison that disables depth/colour mapping while keeping all other parameters fixed.
- The current system relies on hand-tuned HSV thresholds and several maze-sensitive navigation constants.
- Odometry is not corrected by scan matching or loop closure, so long-run map alignment can drift.

3 Conclusion

The project implements an autonomous Webots navigation pipeline that combines encoder/compass odometry, log-odds occupancy mapping, RGB and depth perception, frontier waypoint selection, A* planning, B-spline path smoothing, PID-based look-ahead steering, and recovery logic. The depth and wood-colour processing are used to insert likely elevated obstacles that may be missed by the 2D LiDAR, while green image regions are projected into the grid as hazards.

The recorded results show complete timed runs for Mazes 1, 3, 4, and 5. For Maze 2, only the Start-to-Blue leg is documented, so no full-completion claim is made for that run. Optional mechanisms such as map freezing, breadcrumb retracing, approach-via offsets, and fast wedge detection exist in the code base but were disabled in the uploaded configuration.

Future work should evaluate repeated trials, report success rates, compare the fused system against a LiDAR-only baseline, and improve localization with scan matching or another correction method. Revalidating the smoothed path against the cost map and replacing fixed colour thresholds with more robust perception would also reduce sensitivity to maze-specific tuning.

4 Declaration of AI Usage

We declare the use of AI-assisted tools during this project.

Tools used. GitHub Copilot in Visual Studio Code and a conversational AI assistant.

Purpose and scope.

1. **Code assistance:** autocompletion, debugging support, and discussion while developing mapping, perception, path planning, control, and recovery functions.
2. **Report preparation:** assistance with structure, wording, consistency checking, and document formatting.
3. **Technical discussion:** explanation and review of log-odds mapping, depth-height filtering, A* cost terms, PID control, and parameter tuning.

Declaration. The authors reviewed the generated suggestions against the submitted implementation, corrected unsupported statements, and remain responsible for the design decisions, testing, results, and final report.

A Supplementary Figures

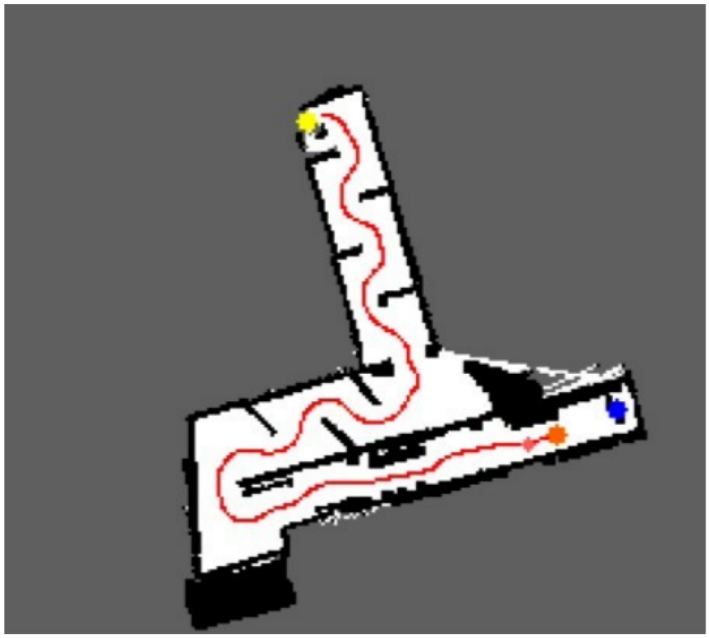


Figure 1: Example live occupancy-grid view. White denotes free cells, grey denotes unknown space, black denotes mapped/inflated obstacles, the red curve is the displayed planned path, the orange marker is the robot, and the blue and yellow markers are stored pillar estimates.

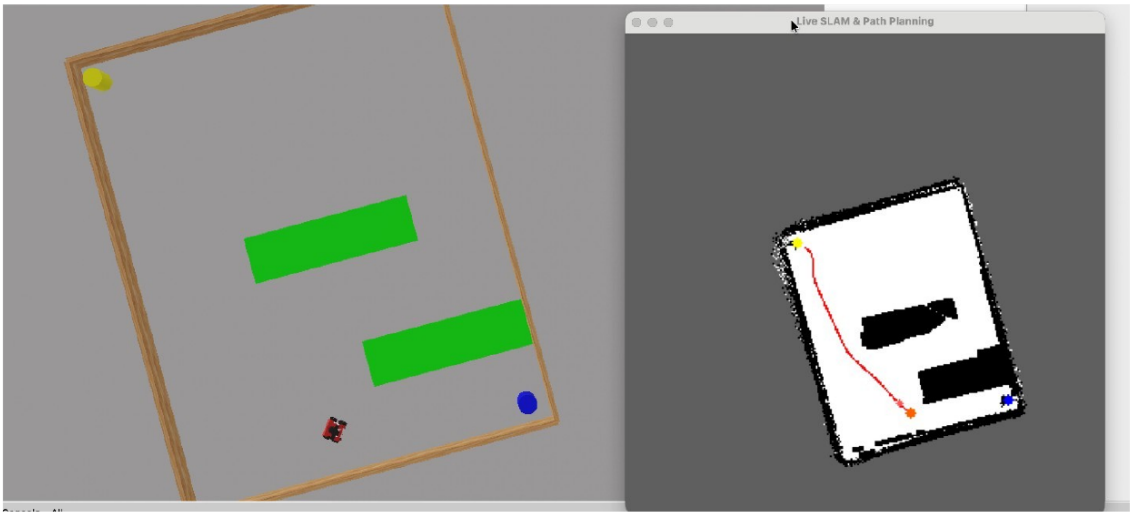


Figure 2: Webots scene and corresponding live map during a run containing green floor regions. In the map window, the robot is orange, pillar estimates are blue and yellow, mapped obstacles are black, and the planned path is red. Green regions are stored as obstacle cells rather than rendered green in the occupancy grid.